

数据结构

线段树模板

```
//区间乘法
#include<bits/stdc++.h>
using namespace std;
#define maxn 1000010
#define ll long long
int p;
ll a[maxn], w[maxn*4];
ll lzy_add[maxn*4], lzy_mul[maxn*4];

void pushup(const int u) {
    w[u] = (w[u<<1] + w[u<<1|1]) % p;
}

void build(const int u, int L, int R) {
    lzy_mul[u] = 1;
    lzy_add[u] = 0;
    if (L == R) {
        w[u] = a[L] % p;
        return;
    }
    int M = (L+R)>>1;
    build(u<<1, L, M);
    build(u<<1|1, M+1, R);
    pushup(u);
}

bool InRange(int L, int R, int l, int r) {
    return (l <= L) && (R <= r);
}

bool OutofRange(int L, int R, int l, int r) {
    return (R < l) || (L > r);
}

void maketag(int u, int L, int R, ll x, int type) {
    if (type == 1) {
        (lzy_mul[u] *= x) %= p;
    }
}
```

```

        (lzy_add[u] *= x) %= p;
        (w[u] *= x) %= p;
    } else {
        (lzy_add[u] += x) %= p;
        (w[u] += (R-L+1) * x) %= p;
    }
}

```

```

void pushdown(int u, int L, int R) {
    int M = (L+R)>>1;
    // 左子节点
    maketag(u<<1, L, M, lzy_mul[u], 1);
    maketag(u<<1, L, M, lzy_add[u], 2);
    // 右子节点
    maketag(u<<1|1, M+1, R, lzy_mul[u], 1);
    maketag(u<<1|1, M+1, R, lzy_add[u], 2);
    lzy_mul[u] = 1;
    lzy_add[u] = 0;
}

```

```

11 query(int u, int L, int R, int l, int r) {
    if (InRange(L, R, l, r)) {
        return w[u];
    } else if (!OutofRange(L, R, l, r)) {
        int M = (L+R)>>1;
        pushdown(u, L, R);
        return (query(u<<1, L, M, l, r) + query(u<<1|1, M+1, R, l, r)) % p;
    } else {
        return 0;
    }
}

```

```

void update(int u, int L, int R, int l, int r, ll x, int type) {
    if (InRange(L, R, l, r)) {
        maketag(u, L, R, x, type);
    } else if (!OutofRange(L, R, l, r)) {
        int M = (L+R)>>1;
        pushdown(u, L, R);
        update(u<<1, L, M, l, r, x, type);
        update(u<<1|1, M+1, R, l, r, x, type);
        pushup(u);
    }
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll n, m;
    cin >> n >> m >> p;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    build(1, 1, n);
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1 || op == 2) {
            ll k;
            cin >> k;
            update(1, 1, n, x, y, k, op);
        } else {
            cout << query(1, 1, n, x, y) << endl;
        }
    }
    return 0;
}

```

主席树（可持久化线段树）

```
//对于单点历史修改与查询
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int maxn=(1e6+1);
int top,root[maxn],arr[maxn],n,m,cnt=0;
struct kkk{
    int l,r,val;
} a[maxn*30];
int build(int l,int r){
    int rt=++cnt;
    if(l==r)a[rt].val=arr[l];
    else {
        int mid=(l+r)>>1;
        a[rt].l=build(l,mid);
        a[rt].r=build(mid+1,r);
    }
    return rt;
}
int update(int ji,int jv,int l,int r,int i)//单点修改
{
    int rt=++cnt;
    a[rt].l=a[i].l;
    a[rt].r=a[i].r;
    if(l==r) a[rt].val=jv;
    else {
        int mid=(l+r)>>1;
        if(ji<=mid)a[rt].l=update(ji,jv,l,mid,a[rt].l);
        else a[rt].r=update(ji,jv,mid+1,r,a[rt].r);
    }
    return rt;
}
int query(int ji,int l,int r,int i){
    if(l==r) return a[i].val;
    int mid=(l+r)>>1;
    if(ji<=mid) return query(ji,l,mid,a[i].l);
    else return query(ji,mid+1,r,a[i].r);
}
signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

cin>>n>>m;
for(int i=1;i<=n;i++){
    cin>>arr[i];
}
root[0]=build(1,n);
for(int i=1,version,op,x,v;i<=m;i++){
    cin>>version>>op>>x;
    if(op==1){
        cin>>v;
        root[i]=update(x,v,1,n,root[version]);
    }else {
        root[i]=root[version];
        cout<<query(x,1,n,root[i])<<endl;
    }
}
}

```

//这个更加重要

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const int maxn=200001;
struct Node{
    int ls,rs;
    int cnt;
}tr[maxn*20];
int cur,rt[maxn];
void init(){
    cur=0;
}
inline void push_up(int o){
    tr[o].cnt=tr[tr[o].ls].cnt+tr[tr[o].rs].cnt;
}
int build(int l,int r){
    int k=cur++;
    if (l==r) {
        tr[k].cnt=0;
        return k;
    }
    int mid=(l+r)>>1;
    tr[k].ls=build(l,mid);
    tr[k].rs=build(mid+1,r);
    push_up(k);
    return k;
}
int update(int o,int l,int r,int pos,int val){
    int k=cur++;
    tr[k]=tr[o];
    if (l==pos&&r==pos){
        tr[k].cnt+=val;
        return k;
    }
    int mid=(l+r)>>1;
    if (pos<=mid) tr[k].ls=update(tr[o].ls,l,mid,pos,val);
    else tr[k].rs=update(tr[o].rs,mid+1,r,pos,val);
    push_up(k);
    return k;
}
```

```

int query(int l,int r,int o,int v,int kth){
    if (l==r) return l;
    int mid=(l+r)>>1;
    int res=tr[tr[v].ls].cnt-tr[tr[o].ls].cnt;
    if (kth<=res) return query(l,mid,tr[o].ls,tr[v].ls,kth);
    else return query(mid+1,r,tr[o].rs,tr[v].rs,kth-res);
}

int b[maxn];
int sortb[maxn];
int main()
{
    int n,m;
    // int T;
    //scanf("%d",&T);
    //while (T--){
    while (~scanf("%d%d",&n,&m)){
        init();
        for (int i=1;i<=n;i++){
            scanf("%d",&b[i]);
            sortb[i]=b[i];
        }
        sort(sortb+1,sortb+1+n);
        int cnt=1;
        for (int i=2;i<=n;i++){
            if (sortb[i]!=sortb[cnt]){
                sortb[++cnt]=sortb[i];
            }
        }
        rt[0]=build(1,cnt);
        for (int i=1;i<=n;i++){
            int p=lower_bound(sortb+1,sortb+cnt+1,b[i])-sortb;
            rt[i]=update(rt[i-1],1,cnt,p,1);
        }
        for (int i=0;i<m;i++){
            int a,b,k;
            scanf("%d%d%d",&a,&b,&k);
            int idx=query(1,cnt,rt[a-1],rt[b],k);
            printf("%d\n",sortb[idx]);
        }
    }
    return 0;
}

```

树状数组

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
int b[500010];
int lowbit(int x){
    return x&(-x);
}
int count(int p){
    int sum=0;
    while(p){
        sum+=b[p];
        p-=lowbit(p);
    }
    return sum;
}
void add(int p,int x,int n){
    while(p<=n){
        b[p]+=x;
        p+=lowbit(p);
    }
}
void solve() {
    int n,m,prev=0;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int tmp;
        cin>>tmp;
        add(i,tmp-prev,n); // 差分建树
        prev=tmp;
    }
    while(m--){
        int choice;
        cin>>choice;
        if(choice==1){
            int x,y,k;
            cin>>x>>y>>k;
            add(x,k,n);
            add(y+1,-k,n); // 差分
        }
        else{
```



```
    int x;
    cin>>x;
    cout<<count(x)<<endl;
}
}
}
signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

开点线段树

```
//适用于范围很大，但是操作较少
//用于减小空间复杂度
//如果操作过多，则与没有优化相同，因此可能被卡
//在线段树的合并与分裂中不可替代，树链剖分
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=100001;
ll le[maxn],ri[maxn],sum[maxn],add[maxn],cnt;
void up(ll i, ll l, ll r){
    sum[i]=sum[l]+sum[r];
}
void lazy(ll i,ll v, ll n){
    sum[i]+=v*n;
    add[i]+=v;
}
void down(ll i, ll ln, ll rn){
    if(add[i]!=0){
        if(le[i]==0){
            le[i]=++cnt;
        }
        if(ri[i]==0){
            ri[i]=++cnt;
        }
        lazy(le[i],add[i],ln);
        lazy(ri[i],add[i],rn);
        add[i]=0;
    }
}
void Add(ll jl,ll jr,ll v,ll l,ll r,ll i){
    if(jl<=l&&r<=jr){
        lazy(i,v,r-l+1);
        return;
    }

    ll mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    if(jl<=mid){
        if(le[i]==0){
            le[i]=++cnt;
        }
    }
}
```

```

        Add(jl,jr,v,l,mid,le[i]);
    }
    if(jr>mid){
        if(ri[i]==0){
            ri[i]=++cnt;
        }
        Add(jl,jr,v,mid+1,r,ri[i]);
    }

    up(i,le[i],ri[i]);
}
ll query(ll jl,ll jr,ll l, ll r,ll i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    ll mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    ll ans=0;
    if(jl<=mid){
        if(le[i]!=0){
            ans+=query(jl,jr,l,mid,le[i]);
        }
    }
    if(jr>mid){
        if(ri[i]!=0){
            ans+=query(jl,jr,mid+1,r,ri[i]);
        }
    }
    return ans;
}
int main(){
    int n,m;
    cin>>n>>m;
    cnt=1;
    while(m--){
        int opt;
        cin>>opt;
        if(opt==1){
            int l,r,v;
            cin>>l>>r>>v;
            Add(l,r,v,1,n,1);
        }
        else {

```

```
    int l,r;  
    cin>>l>>r;  
    cout<<query(l,r,1,n,1)<<endl;  
}  
  
}  
  
}
```

权值线段树

权值线段树模板

```
#define lson pos<<1
#define rson pos<<1|1
void build(int pos,int l,int r)
{
    int mid=(l+r)>>1;
    if(l==r)
    {
        tree[pos]=a[l];//a[l]表示数l有多少个
        return;
    }
    build(lson,l,mid);
    build(rson,mid+1,r);
    tree[pos]=tree[lson]+tree[rson];
}
void update(int pos,int l,int r,int k,int cnt)//表示数k的个数多cnt个
{
    int mid=(l+r)>>1;
    if(l==r)
    {
        tree[pos]+=cnt;
        return;
    }
    if(k<=mid)
        update(lson,l,mid,k,cnt);
    else
        update(rson,mid+1,r,k,cnt);
    tree[pos]=tree[lson]+tree[rson];
}
int query(int pos,int l,int r,int k)//查询数k有多少个
{
    int mid=(l+r)>>1;
    if(l==r)
        return tree[pos];
    if(k<=mid)
        return query(lson,l,mid,k);
    else
        return query(rson,mid+1,r,k);
}
int kth(int pos,int l,int r,int k)//查询第k大值是多少
{

```

```
int mid=(l+r)>>1;
if(l==r)
    return l;
if(k<mid)
    return kth(rson,mid+1,r,k);
else
    return kth(lson,l,mid,k-mid);
}
```

线段树上最值操作

//基本的实现

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1000001;
const int minest=-1000001;
long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],cnt[maxn<<2];
void up(int i){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    mx[i]=max(mx[l],mx[r]);
    if(mx[l]>mx[r]){
        cnt[i]=cnt[l];
        sem[i]=max(sem[l],mx[r]);
    }else if(mx[r]>mx[l]){
        cnt[i]=cnt[r];
        sem[i]=max(sem[r],mx[l]);
    }else {
        sem[i]=max(sem[l],sem[r]);
        cnt[i]=cnt[l]+cnt[r];
    }
}
void Lazy(int i,int v){
    if(v<mx[i]){
        sum[i]-=(mx[i]-v)*cnt[i];
        mx[i]=v;
    }
}
void down(int i){
    Lazy(i<<1,mx[i]);
    Lazy(i<<1|1,mx[i]);
}
void setmin(int jl,int jr,int v,int l,int r,int i){
    if(v>mx[i]) return;
    if(jl<=l&&r<=jr&&v>sem[i]){
        Lazy(i,v);
        return;
    }else {
        down(i);
        int mid=(l+r)>>1;
        if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
```

```

        if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}

long long querymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return mx[i];
    }
    down(i);
    int mid=(l+r)>>1;
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);
    return max(lm,rm);
}

long long querysum(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    down(i);
    int mid=(l+r)>>1;
    long long ans=0;
    if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
    if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
    return ans;
}

void build(int l,int r,int i){
    if(l==r){
        sum[i]=arr[l];
        mx[i]=arr[l];
        sem[i]=minest;
        cnt[i]=1;
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i);
}

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin>>t;

```



```

while(t--){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>arr[i];
    }
    build(1,n,1);
    while(m--){
        int opt;
        cin>>opt;
        if(opt==0){
            int l,r;
            long long mi;
            cin>>l>>r>>mi;
            setmin(l,r,mi,1,n,1);
        }else if(opt==1){
            int l,r;
            cin>>l>>r;
            cout<<querymax(l,r,1,n,1)<<endl;
        }else {
            int l,r;
            cin>>l>>r;
            cout<<querysum(l,r,1,n,1)<<endl;
        }
    }
}
}

```

//如果在上一题的基础上补充范围增加，则代码如下

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1000001;
const int minest=-1000001;
long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],cnt[maxn<<2],maxadd[maxn<<2],other;
void up(int i){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    mx[i]=max(mx[l],mx[r]);
    if(mx[l]>mx[r]){
        cnt[i]=cnt[l];
        sem[i]=max(sem[l],mx[r]);
    }else if(mx[r]>mx[l]){
        cnt[i]=cnt[r];
        sem[i]=max(sem[r],mx[l]);
    }else {
        sem[i]=max(sem[l],sem[r]);
        cnt[i]=cnt[l]+cnt[r];
    }
}
void Lazy(int i,int n,long long maxaddv,long long otheraddv){
    sum[i]+=maxaddv*cnt[i]+otheraddv*(n-cnt[i]);
    mx[i]+=maxaddv;
    sem[i]+=sem[i]==minest?0:otheraddv;
    maxadd[i]+=maxaddv;
    otheradd[i]+=otheraddv;
}
void down(int i,int ln,int rn){
    int l=i<<1;
    int r=i<<1|1;
    int temp=max(mx[l],mx[r]);
    if(mx[l]==temp){
        Lazy(l,ln,maxadd[i],otheradd[i]);
    }else Lazy(l,ln,otheradd[i],otheradd[i]);
    if(mx[r]==temp) Lazy(r,rn,maxadd[i],otheradd[i]);
    else Lazy(r,rn,otheradd[i],otheradd[i]);
    maxadd[i]=0;
    otheradd[i]=0;
}
void setmin(int jl,int jr,int v,int l,int r,int i){
    if(v>mx[i]) return;
```

```

    if(jl<=l&&r<=jr&&v>sem[i]){
        Lazy(i,r-l+1,v-mx[i],0);
        return;
    }else {

        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
        if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}

void add(int jl,int jr,int v,int l,int r,int i){
    if(jl<=l&&r<=jr){
        Lazy(i,r-l+1,v,v);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) add(jl,jr,v,l,mid,i<<1);
        if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
        up(i);
    }
}

long long querymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return mx[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);
    return max(lm,rm);
}

long long querysum(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long ans=0;

```

```

    if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
    if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
    return ans;
}

void build(int l,int r,int i){
    if(l==r){
        sum[i]=arr[l];
        mx[i]=arr[l];
        sem[i]=minest;
        cnt[i]=1;
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i);
}

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin>>t;
    while(t--){
        int n,m;
        cin>>n>>m;
        for(int i=1;i<=n;i++){
            cin>>arr[i];
        }
        build(1,n,1);
        while(m--){
            int opt;
            cin>>opt;
            if(opt==0){
                int l,r;
                long long mi;
                cin>>l>>r>>mi;
                setmin(l,r,mi,1,n,1);
            }else if(opt==1){
                int l,r;
                cin>>l>>r;
                cout<<querymax(l,r,1,n,1)<<endl;
            }else {
                int l,r;
                cin>>l>>r;
            }
        }
    }
}

```

```
        cout<<querysum(1,r,1,n,1)<<endl;
    }
}
}
```

//于此基础上如果还要增加一个查询历史最大值的操作

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=500010;
const long long minest=-1e18;
long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],cnt[maxn<<2],maxadd[maxn<<2],other;
long long maxhistory[maxn<<2],maxaddtop[maxn<<2],otheraddtop[maxn<<2];
void up(int i){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    maxhistory[i]=max(maxhistory[l],maxhistory[r]);
    mx[i]=max(mx[l],mx[r]);
    if(mx[l]>mx[r]){
        cnt[i]=cnt[l];
        sem[i]=max(sem[l],mx[r]);
    }else if(mx[r]>mx[l]){
        cnt[i]=cnt[r];
        sem[i]=max(sem[r],mx[l]);
    }else {
        sem[i]=max(sem[l],sem[r]);
        cnt[i]=cnt[l]+cnt[r];
    }
}
//maxaddvtop:最大值到过的最大提升幅度
//otheraddvtop: 其他值到过的最大提升幅度
void Lazy(int i,int n,long long maxaddv,long long otheraddv,long long maxaddvtop,long long otheraddvtop){
    maxhistory[i]=max(maxhistory[i],mx[i]+maxaddvtop);
    maxaddtop[i]=max(maxaddtop[i],maxadd[i]+maxaddvtop);
    otheraddtop[i]=max(otheraddtop[i],otheradd[i]+otheraddvtop);
    sum[i]+=maxaddv*cnt[i]+otheraddv*(n-cnt[i]);
    mx[i]+=maxaddv;
    sem[i]+=sem[i]==minest?0:otheraddv;
    maxadd[i]+=maxaddv;
    otheradd[i]+=otheraddv;
}
void down(int i,int ln,int rn){
    int l=i<<1;
    int r=i<<1|1;
    int temp=max(mx[l],mx[r]);
    if(mx[l]==temp){
        Lazy(l,ln,maxadd[i],otheradd[i],maxaddtop[i],otheraddtop[i]);
    }
```

```

}else Lazy(l,ln,otheradd[i],otheradd[i],otheraddtop[i],otheraddtop[i]);
if(mx[r]==temp) Lazy(r,rn,maxadd[i],otheradd[i],maxaddtop[i],otheraddtop[i]);
else Lazy(r,rn,otheradd[i],otheradd[i],otheraddtop[i],otheraddtop[i]);
maxadd[i]=0;
otheradd[i]=0;
maxaddtop[i]=0;
otheraddtop[i]=0;
}

void setmin(int jl,int jr,long long v,int l,int r,int i){
    if(v>mx[i]) return;
    if(jl<=l&&r<=jr&&v>sem[i]){
        Lazy(i,r-l+1,v-mx[i],0,v-mx[i],0);
        return;
    }else {

        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
        if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}

void add(int jl,int jr,long long v,int l,int r,int i){
    if(jl<=l&&r<=jr){
        Lazy(i,r-l+1,v,v,v,v);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) add(jl,jr,v,l,mid,i<<1);
        if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
        up(i);
    }
}

long long querymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return mx[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);

```

```

        return max(lm,rm);
    }
long long queryhistorymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return maxhistory[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=queryhistorymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=queryhistorymax(jl,jr,mid+1,r,i<<1|1);
    return max(lm,rm);
}
long long querysum(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long ans=0;
    if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
    if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
    return ans;
}
void build(int l,int r,int i){
    maxadd[i]=0;
    otheradd[i]=0;
    maxaddtop[i]=0;
    otheraddtop[i]=0;
    if(l==r){
        sum[i]=arr[l];
        mx[i]=arr[l];
        sem[i]=minest;
        cnt[i]=1;
        maxhistory[i]=arr[l];
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i);
}

```



```

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    //cin>>t;
    //while(t--){
        int n,m;
        cin>>n>>m;
        for(int i=1;i<=n;i++){
            cin>>arr[i];
        }
        build(1,n,1);
        while(m--){
            int opt;
            cin>>opt;
            if(opt==1){
                int l,r;
                long long v;
                cin>>l>>r>>v;
                add(1,r,v,1,n,1);
            }else if(opt==2){
                long long l,r,v;
                cin>>l>>r>>v;
                setmin(1,r,v,1,n,1);
            }else if (opt==3){
                int l,r;
                cin>>l>>r;
                cout<<querysum(1,r,1,n,1)<<endl;
            }else if(opt==4){
                int l,r;
                cin>>l>>r;
                cout<<querymax(1,r,1,n,1)<<endl;
            }else {
                int l,r;
                cin>>l>>r;
                cout<<queryhistorymax(1,r,1,n,1)<<endl;
            }
        }
    }
}

```

//区间最值操作经典题 hdu 5306

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
const int maxn=500010;
```

```
const long long minest=-1e18;
```

```
long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],cnt[maxn<<2],maxadd[maxn<<2],other;
```

```
long long maxhistory[maxn<<2],maxaddtop[maxn<<2],otheraddtop[maxn<<2];
```

```
void up(int i){
```

```
    int l=i<<1;
```

```
    int r=i<<1|1;
```

```
    sum[i]=sum[l]+sum[r];
```

```
    maxhistory[i]=max(maxhistory[l],maxhistory[r]);
```

```
    mx[i]=max(mx[l],mx[r]);
```

```
    if(mx[l]>mx[r]){
```

```
        cnt[i]=cnt[l];
```

```
        sem[i]=max(sem[l],mx[r]);
```

```
    }else if(mx[r]>mx[l]){
```

```
        cnt[i]=cnt[r];
```

```
        sem[i]=max(sem[r],mx[l]);
```

```
    }else {
```

```
        sem[i]=max(sem[l],sem[r]);
```

```
        cnt[i]=cnt[l]+cnt[r];
```

```
    }
```

```
}
```

//maxaddvtop:最大值到过的最大提升幅度

//otheraddvtop: 其他值到过的最大提升幅度

```
void Lazy(int i,int n,long long maxaddv,long long otheraddv,long long maxaddvtop,long long other
```

```
    maxhistory[i]=max(maxhistory[i],mx[i]+maxaddvtop);
```

```
    maxaddtop[i]=max(maxaddtop[i],maxadd[i]+maxaddvtop);
```

```
    otheraddtop[i]=max(otheraddtop[i],otheradd[i]+otheraddvtop);
```

```
    sum[i]+=maxaddv*cnt[i]+otheraddv*(n-cnt[i]);
```

```
    mx[i]+=maxaddv;
```

```
    sem[i]+=sem[i]==minest?0:otheraddv;
```

```
    maxadd[i]+=maxaddv;
```

```
    otheradd[i]+=otheraddv;
```

```
}
```

```
void down(int i,int ln,int rn){
```

```
    int l=i<<1;
```

```
    int r=i<<1|1;
```

```
    int temp=max(mx[l],mx[r]);
```

```
    if(mx[l]==temp){
```

```
        Lazy(l,ln,maxadd[i],otheradd[i],maxaddtop[i],otheraddtop[i]);
```

```

}else Lazy(l,ln,otheradd[i],otheradd[i],otheraddtop[i],otheraddtop[i]);
if(mx[r]==temp) Lazy(r,rn,maxadd[i],otheradd[i],maxaddtop[i],otheraddtop[i]);
else Lazy(r,rn,otheradd[i],otheradd[i],otheraddtop[i],otheraddtop[i]);
maxadd[i]=0;
otheradd[i]=0;
maxaddtop[i]=0;
otheraddtop[i]=0;
}

void setmin(int jl,int jr,long long v,int l,int r,int i){
    if(v>mx[i]) return;
    if(jl<=l&&r<=jr&&v>sem[i]){
        Lazy(i,r-l+1,v-mx[i],0,v-mx[i],0);
        return;
    }else {

        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
        if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}

void add(int jl,int jr,long long v,int l,int r,int i){
    if(jl<=l&&r<=jr){
        Lazy(i,r-l+1,v,v,v,v);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) add(jl,jr,v,l,mid,i<<1);
        if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
    }
    up(i);
}

long long querymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return mx[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);

```

```

    return max(lm,rm);
}
long long queryhistorymax(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return maxhistory[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long lm=minest,rm=minest;
    if(jl<=mid) lm=queryhistorymax(jl,jr,l,mid,i<<1);
    if(jr>mid) rm=queryhistorymax(jl,jr,mid+1,r,i<<1|1);
    return max(lm,rm);
}
long long querysum(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }

    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    long long ans=0;
    if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
    if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
    return ans;
}
void build(int l,int r,int i){
    maxadd[i]=0;
    otheradd[i]=0;
    maxaddtop[i]=0;
    otheraddtop[i]=0;
    if(l==r){
        sum[i]=arr[l];
        mx[i]=arr[l];
        sem[i]=minest;
        cnt[i]=1;
        maxhistory[i]=arr[l];
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i);
}

```

```

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>arr[i];
    }
    build(1,n,1);
    while(m--){
        int opt;
        cin>>opt;
        if(opt==1){
            int l,r;
            long long v;
            cin>>l>>r>>v;
            add(1,r,v,1,n,1);
        }else if(opt==2){
            long long l,r,v;
            cin>>l>>r>>v;
            setmin(1,r,v,1,n,1);
        }else if (opt==3){
            int l,r;
            cin>>l>>r;
            cout<<querysum(1,r,1,n,1)<<endl;
        }else if(opt==4){
            int l,r;
            cin>>l>>r;
            cout<<querymax(1,r,1,n,1)<<endl;
        }else {
            int l,r;
            cin>>l>>r;
            cout<<queryhistorymax(1,r,1,n,1)<<endl;
        }
    }
}

```

线段树上序列操作

//线段树上的序列操作，稍难一点

//求连续1的最长子列，有点难度，容易打错

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
int arr[maxn],sum[maxn<<2],pre0[maxn<<2],len0[maxn<<2],suf0[maxn<<2],pre1[maxn<<2],len1[maxn<<2];
int lazy[maxn<<2];
bool update[maxn<<2];
bool rever[maxn<<2];
void up(int i,int ln,int rn){
    int l=i<<1;
    int r=i<<1|1;
    sum[i]=sum[l]+sum[r];
    len0[i]=max(max(len0[l],len0[r]),suf0[l]+pre0[r]);
    pre0[i]=len0[l]<ln?pre0[l]:pre0[l]+pre0[r];
    suf0[i]=len0[r]<rn?suf0[r]:suf0[l]+suf0[r];
    len1[i]=max(max(len1[l],len1[r]),suf1[l]+pre1[r]);
    pre1[i]=len1[l]<ln?pre1[l]:pre1[l]+pre1[r];
    suf1[i]=len1[r]<rn?suf1[r]:suf1[l]+suf1[r];
}
void updateLazy(int i,int v,int n){
    sum[i]=v*n;
    pre0[i]=suf0[i]=len0[i]=v==0?n:0;
    pre1[i]=suf1[i]=len1[i]=v==1?n:0;
    lazy[i]=v;
    update[i]=true;
    rever[i]=false;
}
void reverseLazy(int i,int n){
    sum[i]=n-sum[i];
    swap(pre0[i],pre1[i]);
    swap(len0[i],len1[i]);
    swap(suf0[i],suf1[i]);
    rever[i]=!rever[i];
}
void down(int i,int ln,int rn){
    if(update[i]){
        updateLazy(i<<1,lazy[i],ln);
        updateLazy(i<<1|1,lazy[i],rn);
        update[i]=false;
    }
```

```

    }
    if(rever[i]){
        reverseLazy(i<<1,ln);
        reverseLazy(i<<1|1,rn);
        rever[i]=false;
    }
}

void build(int l,int r,int i){
    if(l==r){
        sum[i]=arr[l];
        pre0[i]=arr[l]?0:1;
        suf0[i]=arr[l]?0:1;
        len0[i]=arr[l]?0:1;
        pre1[i]=arr[l]?1:0;
        suf1[i]=arr[l]?1:0;
        len1[i]=arr[l]?1:0;
        return;
    }
    int mid=(l+r)>>1;
    build(l,mid,i<<1),build(mid+1,r,i<<1|1);
    up(i,mid-l+1,r-mid);
    rever[i]=0;
    update[i]=0;
}

void Update(int jl,int jr,int v,int l,int r,int i){
    if(jl<=l&&r<=jr){
        updateLazy(i,v,r-l+1);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) Update(jl,jr,v,l,mid,i<<1);
        if(jr>mid) Update(jl,jr,v,mid+1,r,i<<1|1);
        up(i,mid-l+1,r-mid);
    }
}

void Reverse(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        reverseLazy(i,r-l+1);
    }else {
        int mid=(l+r)>>1;
        down(i,mid-l+1,r-mid);
        if(jl<=mid) Reverse(jl,jr,l,mid,i<<1);
        if(jr>mid) Reverse(jl,jr,mid+1,r,i<<1|1);
    }
}

```

```

        up(i,mid-l+1,r-mid);
    }
}

tuple<int,int,int> querylength(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return make_tuple(len1[i],pre1[i],suf1[i]);
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    if(jr<=mid) return querylength(jl,jr,l,mid,i<<1);
    if(jl>mid) return querylength(jl,jr,mid+1,r,i<<1|1);
    tuple<int,int,int> l1=querylength(jl,jr,l,mid,i<<1);
    tuple<int,int,int> r1=querylength(jl,jr,mid+1,r,i<<1|1);
    int llen=get<0>(l1),lpre=get<1>(l1),lsuf=get<2>(l1);
    int rlen=get<0>(r1),rpre=get<1>(r1),rsuf=get<2>(r1);
    int len=max(max(llen,rlen),lsuf+rpre);
    int pre=lpre<mid-max(l,jl)+1?lpre:lpre+rpre;
    int suf=rsuf<min(r,jr)-mid?rsuf:rsuf+lsuf;
    return make_tuple(len,pre,suf);
}

int query(int jl,int jr,int l,int r,int i){
    if(jl<=l&&r<=jr){
        return sum[i];
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    int ans=0;
    if(jl<=mid) ans+=query(jl,jr,l,mid,i<<1);
    if(jr>mid) ans+=query(jl,jr,mid+1,r,i<<1|1);
    return ans;
}

int main(){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>arr[i];
    build(1,n,1);
    while(m--){
        int opt,l,r;
        cin>>opt>>l>>r;
        l+=1;
        r+=1;
        if(opt==0){
            Update(l,r,0,1,n,1);

```



```
    }else if(opt==1){
        Update(1,r,1,1,n,1);
    }else if(opt==2){
        Reverse(1,r,1,n,1);
    }else if(opt==3){
        cout<<query(1,r,1,n,1)<<endl;
    }else {
        tuple<int,int,int>a=querylength(1,r,1,n,1);
        cout<<get<0>(a)<<endl;
    }
}
}
```

非经典线段树

// 这类题目一般是可以看出使用线段树，但是不能满足一般的懒更新的条件
//简单的一般需要进行势能分析，再减枝就可以解决
//例如 洛谷 P4145，此题是开根号，换为取模同理

//P1471 求方差

```
#include<bits/stdc++.h>
using namespace std;

const int maxn=100001;
double a[maxn],sum[maxn<<2],pf[maxn<<2],lazy[maxn<<2];
void up(int i){
    sum[i]=sum[i<<1]+sum[i<<1|1];
    pf[i]=pf[i<<1]+pf[i<<1|1];
}

void Lazy(int i,double v,int n){
    lazy[i]+=v;
    pf[i]+=2*v*sum[i]+v*v*n;
    sum[i]+=v*n;
}

void down(int i,int ln,int rn){
    if(lazy[i]!=0){
        Lazy(i<<1,lazy[i],ln);
        Lazy(i<<1|1,lazy[i],rn);
        lazy[i]=0;
    }
}

void add(int jl,int jr,double v,int l,int r,int i){
    if(jl<=l&&jr>=r){
        Lazy(i,v,r-l+1);
        return;
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    if(jl<=mid){
        add(jl,jr,v,l,mid,i<<1);
    }
    if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
    up(i);
}

double query(int jl,int jr,int jt,int l,int r,int i){
    if(jl<=l&&jr>=r){
```

```

        if(jt){
            return pf[i];
        }else return sum[i];
    }
    int mid=(l+r)>>1;
    down(i,mid-l+1,r-mid);
    double ans=0;
    if(jl<=mid) ans+=query(jl,jr,jt,l,mid,i<<1);
    if(jr>mid) ans+=query(jl,jr,jt,mid+1,r,i<<1|1);
    return ans;
}

void build(int l,int r,int i){
    if(l==r){
        sum[i]=a[l];
        pf[i]=a[l]*a[l];

    }
    else {
        int mid=(l+r)>>1;
        build(l,mid,i<<1);
        build(mid+1,r,i<<1|1);
        up(i);
    }
    lazy[i]=0;
}

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>a[i];
    }
    build(1,n,1);
    while(m--){
        int opt;
        cin>>opt;
        if(opt==1){
            int l,r;
            double v;
            cin>>l>>r>>v;
            add(l,r,v,1,n,1);
        }else if(opt==2){

```

```

    int l,r;
    cin>>l>>r;
    printf("%.4lf\n",query(l,r,0,1,n,1)/(r-l+1));
}else {
    int l,r;
    cin>>l>>r;
    printf("%.4lf\n",query(l,r,1,1,n,1)/(r-l+1)-query(l,r,0,1,n,1)/(r-l+1)*query(l,r,0,1,
}
}
}

```

左偏树

```
//可并堆结构，两个堆的合并时间复杂度为 $\log(n)$ 
//查询操作需要并查集的优化
//P3377,模板,删除一个堆中最小值
#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
int n,m;
int num[maxn],lef[maxn],righ[maxn],dis[maxn];
int father[maxn];
//father并不是代表该节点的父亲节点，而是代表并查集所对应的路径信息
//需要保证，并查集最上方代表节点=左偏树的头，也就是堆顶
void prepare(){
    dis[0]=-1;
    for(int i=1;i<=n;i++){
        lef[i]=0;
        righ[i]=0;
        dis[i]=0;
        father[i]=i;
    }
}
int find(int i){
    father[i]=father[i]==i?i:find(father[i]);
    return father[i];
}
int merge(int i,int j){
    if(i==0||j==0) return i+j;
    //维护小根堆，如果值一样，编号小的节点做头,大根堆同理
    if(num[i]>num[j]||(num[i]==num[j]&& i>j)){
        swap(i,j);
    }
    righ[i]=merge(righ[i],j);
    if(dis[lef[i]]<dis[righ[i]]){
        swap(lef[i],righ[i]);
    }
    dis[i]=dis[righ[i]]+1;
    father[lef[i]]=father[righ[i]]=i;
    return i;
}
//节点i一定是左偏树的头，在左偏树上删除节点i，返回新树的头结点编号
int pop(int i){
    father[lef[i]]=lef[i];
```

```

    father[righ[i]]=righ[i];
    father[i]=merge(lef[i],righ[i]); //保证删除节点后，能够正确再次查询
    lef[i]=righ[i]=dis[i]=0;
    return father[i];
}
int main(){
    cin>>n>>m;
    prepare();
    for(int i=1;i<=n;i++) cin>>num[i];
    for(int i=1;i<=m;i++){
        int op;
        cin>>op;
        if(op==1){
            int l,r;
            cin>>l>>r;
            if(num[l]==-1&&num[r]==-1){
                l=find(l),r=find(r);
                if(l!=r) merge(l,r);
            }
        }else {
            int x;
            cin>>x;
            if(num[x]==-1) cout<<-1<<endl;
            else {
                int ans=find(x);
                cout<<num[ans]<<endl;
                pop(ans);
                num[ans]=-1;
            }
        }
    }
}

```

```

//进一步，删除任意一个节点
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int maxn=2000100;
int n,m;
int num[maxn],father[maxn],lef[maxn],righ[maxn],dis[maxn],up[maxn];
// up数组存储的就是节点真正的父节点
void prepare(){
    dis[0]=-1;
    for(int i=1;i<=n;i++){
        dis[i]=lef[i]=righ[i]=up[i]=0;
        father[i]=i;
    }
}
int find(int i){
    father[i]=father[i]==i?i:find(father[i]);
    return father[i];
}
int merge(int i,int j){
    if(i==0||j==0){
        return i+j;
    }
    //这段代码是维护的大根堆
    if(num[i]<num[j]||(num[i]==num[j]&& i>j)){
        swap(i,j);
    }
    righ[i]=merge(righ[i],j);
    up[righ[i]]=i;//多了一句
    if(dis[lef[i]]<dis[righ[i]]){
        swap(lef[i],righ[i]);
    }
    dis[i]=dis[righ[i]]+1;
    father[lef[i]]=father[righ[i]]=i;
    return i;
}
//删除任意节点，返回整棵树的头节点
//这是主要的改动
int remove(int i){
    int h=find(i);
    father[lef[h]]=lef[h];
    father[righ[h]]=righ[h];
    int s=merge(lef[h],righ[h]);

```

```

int f=up[i];
father[i]=s;
up[s]=f;
if(h!=i){
    father[s]=h;
    if(lef[f]==i) lef[f]=s;
    else righ[f]=s;
    for(int d=dis[s];dis[f]>d+1;f=up[f],d++){// log(n)
        if(dis[lef[f]]<dis[righ[f]]){
            swap(lef[f],righ[f]);
        }
        dis[f]=d+1;
    }
}
up[i]=lef[i]=righ[i]=dis[i]=0;
return father[s];
}

void reduce(int i,int v){//把i对应的减小v
    int h=remove(i);
    num[i]=max((long long)0,num[i]-v);
    father[h]=father[i]=merge(h,i);
}

void solve(){
    int n,m;
    cin>>n>>m;
    prepare();
    for(int i=1;i<=n;i++){
        cin>>num[i];
    }
    for(int i=1;i<=m;i++){
        int op;
        cin>>op;
        if(op==2){
            int x;
            cin>>x;
            reduce(x,num[x]);
        }else if(op==3){
            int x,y;
            cin>>x>>y;
            reduce(find(x),y);
        }else {
            int x,y;
            cin>>x>>y;

```



```
    x=find(x),y=find(y);  
    if(x!=y) merge(x,y);  
}  
}  
}
```